LinkedIn Data Analyst Jobs Workflow - Complete Documentation

📋 Table of Contents

1. Overview
2. Workflow Architecture
3. Node-by-Node Documentation
4. Setup Instructions
5. Configuration Guide
6. Error Handling
7. Troubleshooting
8. Best Practices
9. Advanced Customization

🎯 Overview

Purpose

This workflow automatically fetches Data Analyst job listings from LinkedIn every morning, filters them using AI, and stores relevant jobs in a Google Sheet for easy tracking.

Key Features

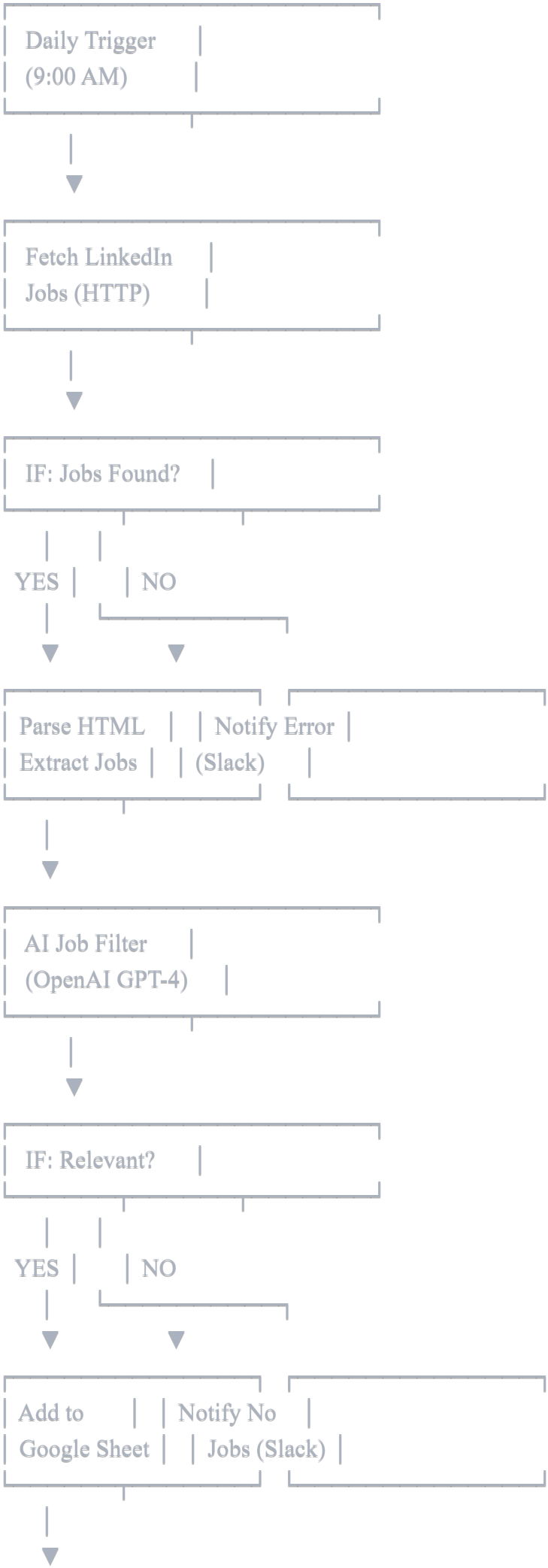
- ✅ Automated daily job fetching at 9:00 AM
- ✅ AI-powered job relevance filtering
- ✅ Automatic Google Sheets updates
- ✅ Slack notifications for success/failure
- ✅ Error-resilient design with graceful fallbacks
- ✅ Configurable search parameters

Use Cases

- Job seekers tracking Data Analyst opportunities
- Recruiters monitoring competitor job postings
- Market researchers analyzing job trends
- Career counselors tracking industry demands

🏗️ Workflow Architecture

High-Level Flow Diagram


Daily Trigger
(9:00 AM)
Fetch LinkedIn
Jobs (HTTP)
IF: Jobs Found?
YES
NO
Parse HTML
Extract Jobs
Notify Error
(Slack)
AI Job Filter
(OpenAI GPT-4)
IF: Relevant?
YES
NO
Add to
Google Sheet
Notify No
Jobs (Slack)

```
┌──────────────────────┐
│ Notify Success       │
│ (Slack)              │
└──────────────────────┘
```

Node Count & Complexity

- **Total Nodes**: 10
- **Trigger Nodes**: 1
- **Action Nodes**: 6
- **Logic Nodes**: 2 (IF conditions)
- **Notification Nodes**: 3

🔧 Node-by-Node Documentation

1. Daily Trigger at 9AM

Type: Schedule Trigger (`n8n-nodes-base.scheduleTrigger`)

Purpose: Automatically executes the workflow every day at 9:00 AM local time.

Configuration:

```json
{
  "cronExpression": "0 9 * * *"
}
```

Cron Breakdown:

- `0` = Minute (at the 0th minute)
- `9` = Hour (9 AM)
- `*` = Every day of the month
- `*` = Every month
- `*` = Every day of the week

Customization Options:

Schedule	Cron Expression	Description
Every 2 hours	`0 */2 * * *`	Runs at minute 0 of every 2nd hour
Twice daily	`0 9,18 * * *`	Runs at 9 AM and 6 PM
Weekdays only	`0 9 * * 1-5`	Runs at 9 AM Monday-Friday
Every Monday	`0 9 * * 1`	Runs at 9 AM every Monday

Testing: Click "Execute Node" to test manually without waiting for the schedule.

---

2. Fetch LinkedIn Jobs

Type: HTTP Request (`n8n-nodes-base.httpRequest`)

Purpose: Retrieves Data Analyst job listings from LinkedIn's job search API.

Configuration:

```json
{
  "method": "GET",
  "url": "https://www.linkedin.com/jobs-guest/jobs/api/seeMoreJobPostings/search",
  "queryParameters": {
    "keywords": "Data Analyst",
    "location": "India",
    "f_TPR": "r518400",
    "start": 0
  },
  "headers": {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
  }
}
```

Query Parameters Explained:

- `keywords`: Job title to search for
- `location`: Geographic location filter
- `f_TPR`: Time filter (`r518400` = past 6 days in seconds)
- `start`: Pagination offset (0 = first page)

Time Filter Values:

Time Range	Value	Calculation
24 hours	r86400	24 × 60 × 60
3 days	r259200	3 × 24 × 60 × 60
6 days	r518400	6 × 24 × 60 × 60
7 days	r604800	7 × 24 × 60 × 60
30 days	r2592000	30 × 24 × 60 × 60

Expected Response:

```json
```

```
{
  "statusCode": 200,
  "body": "<html>...job listings HTML...</html>",
  "headers": {...}
}
```

Error Responses:

- `403`: Rate limited (too many requests)
- `404`: Invalid URL or endpoint changed
- `503`: LinkedIn service unavailable
- `timeout`: Network connection issue

Important Notes: ⚠️ LinkedIn may block automated requests. For production use, consider:

- Using the **Apify LinkedIn Scraper** (more reliable)
- Adding delays between requests
- Using rotating proxies
- Implementing request headers that mimic browser behavior

---

3. Check If Jobs Found

Type: IF Node (`n8n-nodes-base.if`)

Purpose: Validates that the HTTP request successfully returned job data before proceeding.

Configuration:

json

```
{
  "conditions": {
    "combineOperation": "all",
    "conditions": [
      {
        "leftValue": "={{ $json.statusCode }}",
        "rightValue": 200,
        "operator": "equals"
      },
      {
        "leftValue": "={{ $json.body }}",
        "operator": "notEmpty"
      }
    ]
  }
}
```

Logic Flow:

```
IF (statusCode === 200 AND body is not empty)
  → TRUE: Continue to Parse Job Listings
  → FALSE: Send Error Notification
```

Why Both Conditions?:

1. **statusCode = 200**: Confirms successful HTTP request
2. **body not empty**: Confirms actual data was returned

Edge Cases Handled:

- ✅ Network timeout → FALSE → Error notification
- ✅ LinkedIn blocks request → FALSE → Error notification
- ✅ Empty response body → FALSE → Error notification
- ✅ Valid response → TRUE → Continue processing

Debugging: To see what values the IF node is evaluating:

1. Run the workflow
2. Click on "Check If Jobs Found" node
3. View the Input tab to see `$json.statusCode` and `$json.body`

---

4. Parse Job Listings

Type: Code Node (`n8n-nodes-base.code`)

Purpose: Extracts structured job data from LinkedIn's HTML response.

Code Logic:

javascript

```javascript
const html = $input.first().json.body;
const jobs = [];

// Extract job listings using regex
const jobMatches = html.matchAll(/<li[^>]*>(.*?)<\/li>/gs);

for (const match of jobMatches) {
  const jobHtml = match[1];

  // Extract job title
  const titleMatch = jobHtml.match(/<h3[^>]*>([^<]+)<\/h3>/);

  // Extract company name
  const companyMatch = jobHtml.match(/<h4[^>]*>([^<]+)<\/h4>/);

  // Extract location
  const locationMatch = jobHtml.match(/<span[^>]*class="[^"]*location[^"]*"[^>]*>([^<]+)<\/span>/);

  // Extract job URL
  const linkMatch = jobHtml.match(/href="([^"]*\/jobs\/view\/[^"]+)"/);

  // Extract posting date
  const dateMatch = jobHtml.match(/<time[^>]*datetime="([^"]+)"/);

  if (titleMatch && companyMatch) {
    jobs.push({
      job_title: titleMatch[1].trim(),
      company_name: companyMatch[1].trim(),
      location: locationMatch ? locationMatch[1].trim() : 'Not specified',
      date_posted: dateMatch ? dateMatch[1] : new Date().toISOString().split('T')[0],
      job_url: linkMatch ? `https://www.linkedin.com${linkMatch[1]}` : ''
    });
  }
}

return jobs.map(job => ({ json: job }));
```

What This Does:

1. Takes the raw HTML from the HTTP response
2. Uses regex patterns to find job listing elements
3. Extracts specific fields from each job:
    - Job Title
    - Company Name
    - Location
    - Posting Date
    - Job URL
4. Returns an array of structured job objects

Output Format:

```
[
  {
    "json": {
      "job_title": "Senior Data Analyst",
      "company_name": "S&P Global",
      "location": "Hyderabad, India",
      "date_posted": "2025-10-25",
      "job_url": "https://www.linkedin.com/jobs/view/1234567890/"
    }
  },
  {
    "json": {
      "job_title": "Data Analyst",
      "company_name": "Amazon",
      "location": "Bangalore, India",
      "date_posted": "2025-10-26",
      "job_url": "https://www.linkedin.com/jobs/view/0987654321/"
    }
  }
]
```

Important Notes: ⚠️ **LinkedIn's HTML structure may change**, causing this parsing to fail. If you see empty results:

1. Inspect the actual HTML response
2. Update the regex patterns accordingly
3. Consider using a more robust HTML parser like Cheerio

Alternative Approach (More Reliable): Instead of regex, use a proper HTML parser:

```javascript
```

```
// Using built-in DOMParser (if available)
const parser = new DOMParser();
const doc = parser.parseFromString(html, 'text/html');
const jobElements = doc.querySelectorAll('li.job-listing');
```

---

5. AI Job Filter

Type: OpenAI Node (`@n8n/n8n-nodes-langchain.openAi`)

Purpose: Uses GPT-4 to analyze each job and determine if it's a relevant Data Analyst position.

Configuration:

```
{
  "resource": "chat",
  "model": "gpt-4o-mini",
  "messages": [
    {
      "role": "system",
      "content": "You are a job filtering assistant. Analyze job listings and determine if they are relevant Data Analyst pos
    },
    {
      "role": "user",
      "content": "Job Title: {{ $json.job_title }}\nCompany: {{ $json.company_name }}\nLocation: {{ $json.location }}
    }
  ],
  "options": {
    "temperature": 0.3,
    "maxTokens": 100
  }
}
```

Why AI Filtering?: Not all jobs with "Data Analyst" in the title are actually data analysis roles:

- ❌ "Quality Analyst" → Customer support role
- ❌ "Business Analyst (Sales)" → Sales-focused, not data
- ✅ "Data Analyst" → Actual data role
- ✅ "Junior Data Scientist" → Relevant alternative title

Model Parameters:

- `temperature: 0.3` → Low creativity, more consistent responses

- `maxTokens: 100` → Limits response length (cost optimization)

Example AI Responses:

```
Input: "Senior Data Analyst at Amazon"
Output: "RELEVANT - This is a core data analyst position at a major tech company."

Input: "Quality Assurance Analyst"
Output: "NOT_RELEVANT - This is a QA testing role, not data analysis."
```

Cost Estimation:

- **Model**: GPT-4o-mini
- **Cost**: ~$0.0001 per job analyzed
- **Daily Cost** (if 50 jobs): ~$0.005/day = $1.50/year
- **Alternative**: Use GPT-3.5-turbo for even lower cost

Credential Setup:

1. Go to OpenAI Platform
2. Create an API key
3. In n8n: Settings → Credentials → OpenAI → Add credentials
4. Paste your API key

---

6. Filter Relevant Jobs

Type: IF Node (`n8n-nodes-base.if`)

Purpose: Routes jobs based on AI's relevance determination.

Configuration:

```
json
```

```
{
  "conditions": {
    "string": [
      {
        "value1": "={{ $json.message.content }}",
        "operation": "contains",
        "value2": "RELEVANT"
      }
    ]
  }
}
```

Logic:

```
IF AI response contains "RELEVANT"
  → TRUE: Add to Google Sheets
  → FALSE: Skip this job
```

Why "contains" instead of "equals"?:

- AI might respond: "RELEVANT - This is a data analyst role"
- Using "contains" catches "RELEVANT" anywhere in the response
- More flexible and error-tolerant

Data Flow:

```
Parse Job Listings → [10 jobs]
        ↓
AI Filter → [10 AI responses]
        ↓
Filter Relevant Jobs
        ↓
[3 RELEVANT] → Google Sheets
[7 NOT_RELEVANT] → Ignored
```

---

7. Add to Google Sheets

Type: Google Sheets Node (`n8n-nodes-base.googleSheets`)

Purpose: Appends relevant jobs to your tracking spreadsheet.

Configuration:

```json
{
  "operation": "append",
  "documentId": "YOUR_GOOGLE_SHEET_ID",
  "sheetName": "Daily_Data_Analyst_Jobs",
  "columns": {
    "Company Name": "={{ $('Parse Job Listings').item.json.company_name }}",
    "Job Title": "={{ $('Parse Job Listings').item.json.job_title }}",
    "Location": "={{ $('Parse Job Listings').item.json.location }}",
    "Date of Publishing": "={{ $('Parse Job Listings').item.json.date_posted }}",
    "Job URL": "={{ $('Parse Job Listings').item.json.job_url }}"
  }
}
```

How to Get Your Sheet ID: Your Google Sheets URL looks like:

```
https://docs.google.com/spreadsheets/d/1A2B3C4D5E6F7G8H9/edit
                                       ^^^^^^^^^^^^^^^^
                                       This is your Sheet ID
```

Sheet Structure: Your Google Sheet should have these column headers:

Company Name	Job Title	Location	Date of Publishing	Job URL
Amazon	Data Analyst	Bangalore	2025-10-29	https://...

Expression Breakdown:

```javascript
{{ $('Parse Job Listings').item.json.company_name }}
  ↑              ↑  ↑  ↑
  |              |  |  └─ Field name
  |              |  └──────── JSON data
  |              └──────────────── Current item
  └──────────────────────────────────────────────── Reference to node
```

Credential Setup:

1. In n8n: Settings → Credentials → Google Sheets OAuth2
2. Click "Create new credential"
3. Follow OAuth flow to authorize n8n
4. Select the Google account with your tracking sheet

8-10. Notification Nodes

Type: Slack Node (`n8n-nodes-base.slack`)

Purpose: Send alerts for different workflow outcomes.

8. Notify Success

Triggered when a job is successfully added to the sheet.

Message:

✅ Successfully added {{ $json.job_title }} at {{ $json.company_name }} to tracking sheet

Example:

✅ Successfully added Senior Data Analyst at Amazon to tracking sheet

9. Notify Fetch Error

Triggered when LinkedIn request fails.

Message:

⚠️ LinkedIn job fetch failed. Please check the workflow and LinkedIn connection.

When This Happens:

- LinkedIn is down
- Rate limited
- Network issues
- Invalid credentials

10. Notify No Jobs

Triggered when no relevant jobs are found.

Message:

```
ℹ️ No new Data Analyst jobs found in the past 6 days.
```

Slack Setup:

1. Create a Slack workspace (if you don't have one)
2. Create a channel (e.g., `#job-alerts`)
3. In n8n: Settings → Credentials → Slack API
4. Follow instructions to create a Slack app and get OAuth token
5. Invite the bot to your channel: `/invite @n8n-bot`

🛠️ Setup Instructions

Prerequisites

- n8n Cloud account or self-hosted n8n instance (v0.198.0+)
- OpenAI API account with credits
- Google account with Google Sheets access
- Slack workspace (optional but recommended)

Step-by-Step Setup

Step 1: Import the Workflow

1. Copy the complete JSON workflow
2. In n8n, go to **Workflows → Import from File**
3. Paste the JSON or upload the `.json` file
4. Click **Import**

Step 2: Configure OpenAI Credentials

1. Go to [OpenAI Platform]
2. Click **Create new secret key**
3. Copy the API key
4. In n8n: **Settings → Credentials → OpenAI → Add**
5. Paste your API key
6. Click **Save**
7. In the workflow, click **AI Job Filter** node
8. Select your newly created OpenAI credential

Step 3: Configure Google Sheets

1. Create a new Google Sheet
2. Name it "Daily Data Analyst Jobs"
3. Add these column headers in row 1:

```
📋
✓
```

4. Copy the Sheet ID from the URL
5. In n8n: **Settings → Credentials → Google Sheets OAuth2 → Add**
6. Follow OAuth authorization flow
7. In the workflow, click **Add to Google Sheets** node
8. Replace `YOUR_GOOGLE_SHEET_ID` with your actual Sheet ID
9. Select your Google Sheets credential

Step 4: Configure Slack (Optional)

1. Go to Slack API Apps
2. Click **Create New App → From scratch**
3. Name it "n8n Job Alerts"
4. Select your workspace
5. Go to **OAuth & Permissions**
6. Add these scopes:
   - `chat:write`
   - `chat:write.public`
7. Click **Install to Workspace**
8. Copy the **Bot User OAuth Token**
9. In n8n: **Settings → Credentials → Slack API → Add**
10. Paste the token
11. Create a channel (e.g., `#job-alerts`)
12. Invite the bot: `/invite @n8n-job-alerts`
13. In the workflow, configure all Slack nodes with:
    - Your credential
    - Channel name: `#job-alerts`

Step 5: Test the Workflow

1. Click **Execute Workflow** (manual test)
2. Watch each node execute in sequence
3. Check for any errors (red nodes)
4. Verify:
   - ✅ Jobs appear in your Google Sheet
   - ✅ Slack notifications arrive
   - ✅ Only relevant jobs are saved

Step 6: Activate the Workflow

1. Click the **Inactive** toggle at the top
2. It should turn to **Active** (green)
3. The workflow will now run daily at 9 AM

⚙️ Configuration Guide

Changing the Schedule

Edit the Cron Expression in "Daily Trigger at 9AM" node:

Desired Schedule	Cron Expression	Description
Every hour	`0 * * * *`	At minute 0 of every hour
Every 4 hours	`0 */4 * * *`	At 12am, 4am, 8am, 12pm, 4pm, 8pm
Twice daily (9am, 9pm)	`0 9,21 * * *`	At 9am and 9pm
Weekdays only (9am)	`0 9 * * 1-5`	Mon-Fri at 9am
First day of month	`0 9 1 * *`	9am on the 1st of each month

Cron Expression Format:

```
* * * * *
| | | | |
| | | | └─ Day of week (0-7, 0 and 7 are Sunday)
| | | └──── Month (1-12)
| | └─────── Day of month (1-31)
| └──────────── Hour (0-23)
└─────────────── Minute (0-59)
```

Customizing Job Search

In "Fetch LinkedIn Jobs" node, modify query parameters:

```json
{
  "keywords": "Data Analyst",       // Change job title
  "location": "India",              // Change location
  "f_TPR": "r518400",               // Change time range
  "start": 0                        // Pagination offset
}
```

Popular Search Variations:

Job Role	Keyword
Data Analyst	Data Analyst
Data Scientist	Data Scientist
Business Analyst	Business Analyst
Analytics Engineer	Analytics Engineer
BI Developer	Business Intelligence Developer

Location Examples:

- `India` (entire country)
- `Bangalore, India` (specific city)
- `United States` (another country)
- `Remote` (remote jobs)

AI Filtering Customization

Modify the system prompt in "AI Job Filter" node:

Current Prompt:

```
You are a job filtering assistant. Analyze job listings and determine if they are relevant Data Analyst positions.
Return ONLY 'RELEVANT' or 'NOT_RELEVANT' followed by a brief reason.
```

Example Customizations:

Option 1: More Strict Filtering

```
You are a job filtering assistant. ONLY mark jobs as RELEVANT if they explicitly mention:
- Data analysis
- SQL/Python
- Data visualization
- Statistics or analytics
Otherwise mark as NOT_RELEVANT.
```

Option 2: Include Adjacent Roles

```
You are a job filtering assistant. Mark as RELEVANT if the job is:
- Data Analyst
- Data Scientist
- Business Analyst (with analytics focus)
- Analytics Engineer
- BI Developer
Mark as NOT_RELEVANT if it's sales, marketing, or HR analytics.
```

Option 3: Experience Level Filter

```
You are a job filtering assistant. ONLY mark as RELEVANT if:
- Job is entry-level or junior (0-2 years experience)
- Role is Data Analyst, not Data Scientist
Mark senior roles (5+ years) as NOT_RELEVANT.
```

Google Sheets Customization

Change Column Mapping:

In "Add to Google Sheets" node, modify the columns object:

```json
{
  "columns": {
    "Company Name": "={{ $('Parse Job Listings').item.json.company_name }}",
    "Job Title": "={{ $('Parse Job Listings').item.json.job_title }}",
    "Location": "={{ $('Parse Job Listings').item.json.location }}",
    "Date Posted": "={{ $('Parse Job Listings').item.json.date_posted }}",
    "URL": "={{ $('Parse Job Listings').item.json.job_url }}",

    // Add custom columns:
    "Date Added": "={{ $now.toFormat('yyyy-MM-dd') }}",
    "Status": "New",
    "Notes": ""
  }
}
```

Result: Your sheet will now have additional tracking columns.

🚨 Error Handling

Built-in Error Handling

The workflow includes multiple layers of error protection:

1. Continue on Fail

The HTTP Request node has `continueOnFail: true`, meaning:

- If LinkedIn request fails → workflow continues
- Error gets caught by IF node
- Sends notification instead of crashing

2. IF Node Validation

Two IF nodes validate data before processing:

- **Check If Jobs Found**: Prevents parsing empty responses
- **Filter Relevant Jobs**: Ensures only AI-approved jobs are saved

3. Notification System

Three notification paths provide visibility:

- ✅ Success: Job added
- ⚠️ Error: Fetch failed
- ℹ️ Info: No jobs found

Common Errors & Solutions

Error 1: "LinkedIn Request Failed"

Symptoms: Slack message "LinkedIn job fetch failed"

Possible Causes:

1. LinkedIn blocking automated requests
2. Rate limiting (too many requests)
3. Network connectivity issues
4. Invalid search parameters

Solutions:

1. Wait 1-2 hours and try again
2. Use Apify LinkedIn Scraper instead of direct HTTP
3. Add delays between requests
4. Verify your internet connection
5. Check if LinkedIn's API endpoint changed

Alternative Approach: Replace HTTP Request with Apify:

```
Install Apify node
→ Use "LinkedIn Jobs Scraper" actor
→ More reliable, handles rate limits
```

Error 2: "OpenAI Authentication Failed"

Symptoms: AI Filter node shows error

Possible Causes:

1. Invalid API key
2. Expired API key
3. No credits in OpenAI account
4. API key lacks permissions

Solutions:

1. Go to OpenAI Platform → API Keys
2. Verify key is active and not expired
3. Check billing: Platform → Billing → Usage
4. Generate a new API key if needed
5. Update credential in n8n

Error 3: "Google Sheets Permission Denied"

Symptoms: Can't write to sheet

Possible Causes:

1. OAuth token expired
2. Sheet doesn't exist
3. Wrong Sheet ID
4. Sheet name mismatch

Solutions:

1. Re-authenticate Google Sheets credential
2. Verify Sheet ID is correct
3. Check sheet name matches exactly (case-sensitive)
4. Ensure sheet isn't deleted or moved

Error 4: "No Data Returned from Parsing"

Symptoms: Parse node returns empty array

Possible Causes:

1. LinkedIn changed HTML structure
2. Regex patterns don't match
3. Empty search results

Solutions:

1. Inspect actual HTML response
2. Update regex patterns in Parse node
3. Use browser DevTools to inspect LinkedIn's HTML
4. Consider using a proper HTML parser (Cheerio)

Debug Steps:

1. Run workflow
2. Click "Parse Job Listings" node
3. View Input tab → See raw HTML
4. View Output tab → See extracted jobs
5. If output is empty, HTML parsing failed

Error 5: "Slack Message Not Sent"

Symptoms: No Slack notifications

Possible Causes:

1. Bot not invited to channel
2. Wrong channel name
3. Insufficient permissions
4. Token expired

Solutions:

1. In Slack: `/invite @n8n-job-alerts` in your channel
2. Verify channel name in node matches exactly
3. Check bot has `chat:write` permission
4. Re-authenticate Slack credential

🔍 Troubleshooting

Debugging Workflow Execution

Enable Detailed Logging

1. Go to workflow settings (gear icon)
2. Enable "Save execution progress"
3. Set "Save data of successful executions"
4. Run workflow
5. Click "Executions" tab to see detailed logs

Inspect Node Data

For any node:

1. Click the node after execution
2. **Input tab**: See data coming into the node
3. **Output tab**: See data leaving the node
4. **JSON tab**: View raw JSON structure

Test Individual Nodes

1. Select a node
2. Click "Execute Node" (play button)
3. Node runs independently
4. View results immediately

Common Issues & Fixes

Issue: Workflow Doesn't Run on Schedule

Check:

1. Is workflow **Active** (toggle at top)?
2. Is trigger node configured correctly?
3. Check timezone settings (Settings → General)
4. Wait for next scheduled time

Verify Trigger:

```
Click trigger node → Check cron expression
Test manually: Click "Execute Node"
```

Issue: Duplicate Jobs in Sheet

Cause: No deduplication logic

Solution: Add deduplication check

Before Google Sheets node, add Code node:

javascript

```javascript
// Check if job URL already exists in sheet
const existingJobs = await this.helpers.requestWithAuthentication.call(
  this,
  'googleSheetsOAuth2Api',
  {
    method: 'GET',
    url: `https://sheets.googleapis.com/v4/spreadsheets/${sheetId}/values/A:E`
  }
);

const existingUrls = existingJobs.values.map(row => row[4]); // Column E = URL
const newJob = $input.all()[0].json;

if (!existingUrls.includes(newJob.job_url)) {
  return [{ json: newJob }];
} else {
  return []; // Skip if duplicate
}
```

Issue: Too Many/Few Jobs

Too Many: AI marking irrelevant jobs as relevant **Solution**: Make AI prompt more strict

Too Few: AI being too strict **Solution**: Make AI prompt more lenient

Test AI Filtering:

1. Run workflow manually
2. Click "AI Job Filter" node
3. Review Input/Output for each job
4. Adjust system prompt accordingly

Issue: High OpenAI